

BANCO DE 50 PREGUNTAS

DEFENSA ORAL DEL BACKEND

Spring Boot + MVC + Monolito Hexagonal + JPA/Hibernate + PostgreSQL + API REST

VERSION DOCENTE - PREPARACION Y EVALUACION DE DEFENSA INDIVIDUAL

PROPOSITO

Comprobar que el estudiante no solo puede ejecutar su proyecto, sino que comprende y puede defender cada decision, anotacion y linea importante del flujo: desde una solicitud HTTP originada en navegador, Swagger o Postman, hasta el Controller, DTO/record, validacion, caso de uso/servicio, repositorio, JPA/Hibernate, SQL y PostgreSQL; y luego explicar el camino de regreso hasta la respuesta HTTP.

Regla central: una definicion memorizada sin poder ubicarla y justificarla en el propio codigo se considera respuesta incompleta.

1. Criterio pedagogico de la defensa

Este banco adapta al backend la misma filosofia de trabajo del mapa conceptual entregado: evaluar comprension, jerarquia, relaciones tecnicas reales y capacidad de explicar recorridos completos. Cada estudiante debe conectar conceptos con su proyecto concreto, no repetir definiciones aisladas.

Exigencia	Que debe hacer el estudiante
1. Definir	Explicar el concepto con palabras propias, sin leer.
2. Ubicar	Mostrar en que paquete, clase, metodo o anotacion aparece en su proyecto.
3. Recorrer	Explicar que sucede antes y despues de ese punto dentro del flujo.
4. Justificar	Explicar por que se uso asi y que alternativa existia.
5. Predecir	Decir que ocurriria si se elimina, cambia o falla esa pieza.

2. Escala de calificacion sugerida

Cada pregunta puede calificarse de 0 a 2 puntos. Con 50 preguntas, el banco completo equivale a 100 puntos.

Puntaje	Nivel	Criterio
0	No domina	No puede explicarlo, confunde conceptos o no puede localizarlo en su codigo.
1	Parcial	Conoce la idea, pero falla al recorrer el flujo, justificar la decision o relacionarla con su proyecto.
2	Dominio	Explica, localiza, conecta, justifica y predice consecuencias tecnicas con seguridad.

Interpretacion sugerida: 85-100: preparado; 70-84: defendible con vacios; 50-69: preparacion insuficiente; menos de 50: no domina su backend. **Ademas, fallar dos o mas preguntas marcadas como CRITICA debe obligar a una repregunta tecnica adicional.**

3. Forma recomendada de aplicar el examen oral

- Seleccionar 10 a 12 preguntas al azar, pero incluir siempre al menos una de arquitectura, una de MVC/HTTP, una de POST, una de GET/PUT/DELETE, dos de JPA y una de DTO/servicio/repositorio.
- En 3 o 4 preguntas exigir que el estudiante abra el IDE y señale el codigo exacto que esta explicando.
- En al menos una pregunta pedir que dibuje el flujo completo en pizarra: cliente -> Spring MVC -> aplicacion -> persistencia -> PostgreSQL -> respuesta.
- No aceptar respuestas del tipo "Spring lo hace". Debe indicar que componente de Spring interviene y que responsabilidad cumple.
- Cambiar nombres y escenarios para impedir respuestas memorizadas: EntidadPadre/EntidadHija deben sustituirse por las entidades reales del proyecto del estudiante.

4. Bloque A - Arquitectura, Hexagonal, MVC y Spring

Objetivo: comprobar que el estudiante entiende la arquitectura y puede ubicar cada responsabilidad en su propio proyecto.

PREGUNTA 01 - CRITICA

Dibuja la arquitectura real de tu proyecto y explica el recorrido entre sus paquetes.

Debe demostrar:

- Debe identificar, con los nombres reales de su código, donde están dominio/modelo, aplicación/casos de uso o servicios, adaptadores de entrada (REST) y adaptadores de salida (persistencia).
- Debe diferenciar organización física de paquetes de responsabilidad arquitectónica.
- Debe explicar la dirección de las dependencias y evitar respuestas basadas solo en "controller-service-repository".

Repregunta de presión: Si muevo tu Controller al paquete domain, ¿el proyecto puede compilar? Aunque compile, ¿que principio arquitectónico estaría rompiendo y por qué?

PREGUNTA 02

¿Que significa realmente "arquitectura hexagonal" en tu backend y que problema intenta resolver?

Debe demostrar:

- Debe explicar puertos y adaptadores, aislamiento de lógica de negocio y dependencia hacia abstracciones.
- Debe dejar claro que "hexagonal" no significa seis capas ni una forma gráfica literal.
- Debe relacionar el concepto con cambios de tecnología: REST, base de datos, mensajería o pruebas.

Repregunta de presión: Si mañana PostgreSQL se reemplaza por otra persistencia, ¿que partes deberían cambiar y cuáles no?

PREGUNTA 03

Diferencia dominio, aplicación e infraestructura usando clases concretas de tu proyecto.

Debe demostrar:

- Dominio: conceptos/reglas del negocio; aplicación: orquestación de casos de uso; infraestructura: detalles técnicos como HTTP, JPA, PostgreSQL.
- Debe reconocer si su proyecto mezcla responsabilidades y ser capaz de justificarlo.
- No debe confundir una entidad JPA automáticamente con "todo el dominio".

Repregunta de presión: Si tus @Entity están dentro del dominio, ¿que dependencia de framework introduces? ¿Que haría una hexagonal más estricta?

PREGUNTA 04

¿Cuales son tus puertos de entrada y de salida? Señálalos en el código.

Debe demostrar:

- Puerto de entrada: contrato/caso de uso que la aplicación ofrece; puerto de salida: contrato que la aplicación necesita para persistir o integrarse.
- Controller REST actúa como adaptador de entrada; persistencia JPA como adaptador de salida.
- Debe explicar la ventaja de depender de interfaces propias y no de detalles externos.

Repregunta de presión: ¿Tu JpaRepository es un puerto de dominio o un detalle de infraestructura? Defiende tu respuesta según tu implementación.

PREGUNTA 05 - CRITICA

Explica la regla de dependencias del monolito hexagonal y comprueba si tu proyecto la respeta.

Debe demostrar:

- Las capas externas pueden depender de las internas; el núcleo no debería conocer detalles de entrada/salida.
- Debe detectar dependencias circulares o accesos directos Controller -> JpaRepository si contradicen su diseño.
- Debe poder mostrar imports concretos y explicar por qué son aceptables o problemáticos.

Repregunta de presión: Si el servicio importa una clase concreta de PostgreSQL o de un Controller, ¿que acoplamiento aparece?

PREGUNTA 06

¿MVC y arquitectura hexagonal son lo mismo? ¿Como conviven en Spring Boot?

Debe demostrar:

- Debe decir que resuelven problemas diferentes: MVC organiza la interacción web; hexagonal organiza dependencias y fronteras del sistema.
- Spring MVC puede ser un adaptador de entrada dentro de una arquitectura hexagonal.
- No debe presentar MVC como reemplazo de dominio/aplicación.

Repregunta de presión: Ubica exactamente donde termina MVC y donde comienza tu caso de uso.

PREGUNTA 07

En una API REST con Spring MVC, ¿donde estan Model, View y Controller?

Debe demostrar:

- Controller: clase/metodos anotados que reciben la solicitud.
- El "model" de MVC no debe confundirse con la entidad JPA; en REST el resultado suele ser un objeto/DTO que se serializa.
- La "view" puede ser la representacion JSON producida por `HttpMessageConverters/Jackson`, sin `ViewResolver HTML`.

Repregunta de presion: ¿Por que `@RestController` cambia la forma tradicional de pensar la "vista"?

PREGUNTA 08

¿Que hace `@SpringBootApplication` y que ocurre cuando ejecutas `main()`?

Debe demostrar:

- Debe relacionar `@SpringBootApplication` con configuracion, auto-configuracion y component scanning.
- Debe explicar creacion del `ApplicationContext`, descubrimiento de beans y arranque del servidor embebido.
- Debe saber desde que paquete se escanean componentes y por que la ubicacion de la clase principal importa.

Repregunta de presion: Si colocas la clase principal en un paquete demasiado interno, ¿que podria dejar de detectarse?

PREGUNTA 09

Explica IoC, inyeccion de dependencias y por que prefieres inyeccion por constructor.

Debe demostrar:

- Spring crea y conecta beans; la clase declara dependencias en lugar de instanciarlas con `new`.
- Constructor injection hace dependencias explicitas, facilita pruebas y permite campos final.
- Con un solo constructor no suele requerirse `@Autowired`.

Repregunta de presion: ¿Que problema tendrias si dentro del Controller haces `new MiServicio()`?

PREGUNTA 10

Haz una auditoria de todas las anotaciones Spring/JPA que aparecen en tu proyecto: clasificalas y explica para que sirve cada una.

Debe demostrar:

- Debe incluir, si existen: `@RestController`, `@RequestMapping`, `@GetMapping`/`@PostMapping`/`@PutMapping`/`@DeleteMapping`, `@RequestBody`, `@PathVariable`, `@RequestParam`, `@Valid`, `@Service`, `@Repository`, `@Transactional`, `@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`, `@OneToMany`, `@ManyToOne`, `@JoinColumn` y otras usadas.
- Debe diferenciar anotaciones de web, DI, transaccion, validacion y persistencia.
- Debe explicar consecuencias de quitar una anotacion, no solo su definicion.

Repregunta de presion: Elige tres anotaciones de tu codigo: ¿que error o comportamiento distinto apareceria si las eliminas?

5. Bloque B - HTTP, MVC y flujo CRUD end-to-end

Objetivo: obligar al estudiante a recorrer solicitudes reales desde el cliente HTTP hasta la base de datos y volver.

PREGUNTA 11 - CRITICA

Explica el recorrido exacto de una solicitud desde navegador/Postman/Swagger hasta PostgreSQL y de regreso.

Debe demostrar:

- Cliente HTTP -> servidor embebido/filtros -> `DispatcherServlet` -> `HandlerMapping` -> Controller -> deserializacion Jackson -> validacion -> caso de uso/servicio -> puerto/repositorio -> JPA/Hibernate -> JDBC -> PostgreSQL.
- En retorno: resultado SQL -> entidad/modelo -> mapeo DTO -> Jackson -> status/headers/body HTTP -> cliente.
- Debe distinguir pasos que son de su codigo y pasos que pertenecen al framework.

Repregunta de presion: ¿En que punto exacto se convierte JSON en un objeto Java y en que punto se convierte un objeto Java en JSON?

PREGUNTA 12

¿Que diferencia hay entre `@Controller` y `@RestController`? ¿Que aporta `@RequestMapping`?

Debe demostrar:

- `@RestController` combina semantica de controller web con respuesta serializada en el body.
- `@RequestMapping` puede definir ruta base y otras condiciones; los mappings HTTP especializan por metodo.
- Debe señalar las rutas reales del proyecto.

Repregunta de presion: Si cambias `@RestController` por `@Controller` sin `@ResponseBody`, ¿que podria intentar hacer Spring?

PREGUNTA 13**Diferencia @PathVariable, @RequestParam y @RequestBody con endpoints reales.****Debe demostrar:**

- PathVariable identifica un segmento de ruta; RequestParam datos de query; RequestBody contenido del cuerpo.
- Debe explicar como Spring obtiene y convierte cada valor.
- Debe saber que GET normalmente no se diseña alrededor de un body.

Repregunta de presion: *Diseña una ruta para buscar hijos de un padre por estado y explica que iria en path y que en query string.***PREGUNTA 14****¿Swagger, Postman y un navegador "pasan por caminos distintos" dentro de tu backend?****Debe demostrar:**

- Los tres son clientes HTTP; si llaman al mismo endpoint, la ruta interna del backend es esencialmente la misma.
- Swagger UI usa la especificacion OpenAPI para documentar/probar; Postman permite construir peticiones; un navegador puede originar requests directas o desde un frontend.
- Debe diferenciar documentacion/prueba de logica de negocio.

Repregunta de presion: *Si funciona en Swagger pero falla desde React, ¿que revisarias primero: backend, payload, CORS, URL, headers o serializacion? Explica el orden.***PREGUNTA 15 - CRITICA****POST: explica linea por linea que ocurre al crear una EntidadPadre o EntidadHija.****Debe demostrar:**

- Debe recorrer mapping HTTP, Request DTO/record, Jackson, @Valid, Controller, servicio/caso de uso, reglas de negocio, mapper, save(), Hibernate, INSERT SQL, ID generado, Response DTO y status 201 o el definido.
- Debe explicar que pasa con una relacion ManyToOne al asignar el padre.
- Debe diferenciar persistir una nueva entidad de actualizar una ya gestionada.

Repregunta de presion: *Si el id del padre recibido no existe, ¿donde detectas el problema y que status HTTP devolverias?***PREGUNTA 16 - CRITICA****GET por id: explica todo el flujo, incluyendo el caso de que el registro no exista.****Debe demostrar:**

- Debe recorrer @GetMapping, @PathVariable, servicio, repository.findById, Optional, excepcion/not-found, mapeo a Response DTO y serializacion.
- Debe justificar 200 cuando existe y 404 cuando no existe.
- Debe explicar si la relacion uno-a-muchos se carga en ese momento o puede ser lazy.

Repregunta de presion: *¿Que diferencia practica hay entre findById(id).get() y orElseThrow(...)? ¿Cual expresa mejor el caso de uso?***PREGUNTA 17****GET de listado: ¿que cambia respecto al GET por id y que riesgos aparecen con relaciones?****Debe demostrar:**

- Debe explicar findAll o consulta paginada, colecciones, mapeo de multiples resultados y respuesta 200.
- Debe reconocer riesgo de traer demasiados registros y recomendar paginacion cuando corresponde.
- Debe reconocer N+1 o carga involuntaria de relaciones.

Repregunta de presion: *Si tienes 10.000 padres y cada uno 100 hijos, ¿por que devolver findAll() con todo anidado puede ser un desastre?***PREGUNTA 18 - CRITICA****PUT: explica el flujo completo de una actualizacion y por que PUT debe ser idempotente.****Debe demostrar:**

- Debe buscar el recurso existente, validar existencia, aplicar cambios permitidos, preservar identidad, ejecutar reglas y guardar/dirty checking.
- Idempotencia: repetir la misma solicitud debe dejar el recurso en el mismo estado final.
- Debe diferenciar PUT de PATCH y evitar crear una entidad accidentalmente si el id no existe salvo contrato explicito.

Repregunta de presion: *¿Que error aparece si construyes una entidad nueva con el id recibido y haces save() sin verificar existencia?***PREGUNTA 19 - CRITICA****DELETE: explica el flujo completo y que ocurre con los hijos en una relacion uno-a-muchos.****Debe demostrar:**

- Debe recorrer @DeleteMapping, id, comprobacion de existencia, reglas de negocio, delete/deleteById, DELETE SQL y status 204/200 segun contrato.

- Debe explicar FK, cascade y orphanRemoval segun su mapping.
- Debe saber que eliminar padre con hijos puede fallar por integridad referencial si el modelo no lo permite.

Repregunta de presion: Si la base de datos rechaza el DELETE por foreign key, ¿por que no deberias "arreglarlo" simplemente poniendo CascadeType.ALL sin pensar?

PREGUNTA 20

Explica los codigos HTTP 200, 201, 204, 400, 404, 409 y 500 en el contexto de tu API.

Debe demostrar:

- Debe vincular cada codigo a un escenario real.
- Debe diferenciar error del cliente, recurso inexistente, conflicto de negocio/integridad y error interno.
- Debe evitar devolver siempre 200 con un mensaje de error dentro del body.

Repregunta de presion: Dame un ejemplo concreto de tu proyecto donde 409 sea mas correcto que 400.

PREGUNTA 21

¿Que hace @Valid y en que momento se ejecutan @NotNull, @NotBlank, @Size u otras validaciones?

Debe demostrar:

- Se aplica sobre el objeto de entrada y activa Bean Validation antes de entrar plenamente a la logica de negocio del metodo.
- Debe diferenciar validacion sintactica/estructural de reglas de negocio.
- Debe saber que @NotNull y @NotBlank no son equivalentes.

Repregunta de presion: Un correo con formato correcto pero ya registrado: ¿es problema de @Email o regla de negocio?

PREGUNTA 22

¿Como centralizas errores con @RestControllerAdvice y @ExceptionHandler?

Debe demostrar:

- Debe explicar manejo transversal de excepciones y contrato uniforme de error.
- Debe mapear excepciones de dominio/aplicacion a status HTTP sin contaminar cada controller con try/catch repetido.
- Debe mencionar timestamp/codigo/mensaje/campo/ruta si usa un ErrorResponse.

Repregunta de presion: ¿Que ganarias frente a capturar Exception en cada endpoint y devolver 500 manualmente?

PREGUNTA 23

¿Para que sirve @Transactional y donde deberia estar el limite de transaccion?

Debe demostrar:

- Debe explicar atomicidad de una operacion de aplicacion y rollback ante excepciones apropiadas.
- Suele ubicarse en servicio/caso de uso cuando una operacion incluye varias acciones de persistencia.
- Debe relacionar transaccion con persistence context y dirty checking.

Repregunta de presion: Si guardas el padre, luego falla el guardado de un hijo, ¿como evitas quedar con datos a medias?

PREGUNTA 24

¿Que es CORS y por que puede fallar React en el navegador aunque Postman funcione?

Debe demostrar:

- CORS es una politica aplicada por el navegador entre origenes; Postman no aplica las mismas restricciones del browser.
- Debe reconocer origen = esquema + host + puerto y hablar de metodos/headers permitidos.
- Debe evitar confundir CORS con autenticacion o error de base de datos.

Repregunta de presion: Tu backend corre en :8080 y React en :5173. ¿Por que son origenes distintos aunque esten en localhost?

6. Bloque C - JPA, Hibernate, cardinalidad y PostgreSQL

Objetivo: verificar dominio practico de entidades, relaciones, estado de persistencia, SQL e integridad relacional.

PREGUNTA 25

¿Que hacen @Entity y @Table y que significa que una clase sea gestionada por JPA?

Debe demostrar:

- @Entity marca un tipo persistente; @Table permite configurar tabla/esquema/nombre cuando hace falta.
- Hibernate puede mapear instancias gestionadas a filas.
- Debe diferenciar clase Java, entidad persistente y tabla fisica.

Repregunta de presion: Si renombras la tabla en PostgreSQL pero no actualizas el mapping, ¿que esperas que ocurra?

PREGUNTA 26

Explica @Id y @GeneratedValue. ¿Como se genera tu identificador en PostgreSQL?

Debe demostrar:

- Debe identificar la estrategia real: IDENTITY, SEQUENCE, AUTO u otra.
- Debe explicar que el id tiene identidad persistente y no debe tratarse como dato editable ordinario.
- Debe saber que PostgreSQL maneja secuencias/identity segun estrategia y dialecto.

Repregunta de presion: *¿En que momento esperas que la entidad nueva tenga id: antes o despues de persistir? ¿Depende de la estrategia?*

PREGUNTA 27

¿Que puede expresar @Column y que diferencia hay entre esa configuracion y una restriccion real en PostgreSQL?

Debe demostrar:

- Debe explicar nombre, nullable, unique, length, precision/scale segun caso.
- Debe reconocer que el esquema real de produccion debe estar controlado y que la base debe hacer cumplir integridad importante.
- No debe confundir validacion Java con constraint de base de datos.

Repregunta de presion: *Si @Column(nullable=false) dice una cosa pero la tabla existente permite NULL, ¿quien tiene la ultima palabra al ejecutar SQL?*

PREGUNTA 28 - CRITICA

Representa en tablas la cardinalidad uno-a-muchos de tus dos entidades.

Debe demostrar:

- Debe dibujar tabla padre con PK y tabla hija con FK al padre.
- Debe explicar que muchos registros hijos pueden referenciar un mismo padre.
- Debe identificar nombre real de la FK y su nulabilidad.

Repregunta de presion: *¿En que tabla esta fisicamente la foreign key y por que no suele estar en la tabla padre?*

PREGUNTA 29 - CRITICA

Explica @OneToMany y @ManyToOne usando exactamente tu relacion.

Debe demostrar:

- @ManyToOne suele estar en la entidad hija y representa la FK; @OneToMany representa la coleccion desde el padre.
- Debe distinguir cardinalidad Java y cardinalidad relacional.
- Debe saber si su relacion es unidireccional o bidireccional.

Repregunta de presion: *Si solo mantuvieras @ManyToOne en el hijo, ¿la relacion seguiria existiendo en la base? ¿Que perderias en el modelo Java?*

PREGUNTA 30 - CRITICA

¿Cual es el lado propietario de la relacion y que significan mappedBy y @JoinColumn?

Debe demostrar:

- El lado propietario es el que controla la FK; en el mapping bidireccional tipico es el @ManyToOne con @JoinColumn.
- mappedBy apunta al nombre del atributo Java del lado propietario, no al nombre de la columna SQL.
- Debe poder decir el nombre exacto de su campo mappedBy y de su join column.

Repregunta de presion: *Si escribes mappedBy con el nombre de la columna SQL en vez del atributo Java, ¿que tipo de error esperas?*

PREGUNTA 31

Explica cascade y cada CascadeType que realmente utilizas. ¿Por que CascadeType.ALL puede ser peligroso?

Debe demostrar:

- Debe comprender PERSIST, MERGE, REMOVE y otros si los usa; cascade propaga operaciones de entidad padre a relacionadas.
- ALL no debe usarse por comodidad si REMOVE no es deseable.
- Debe relacionar cascade con el ciclo de vida de negocio de padre/hijos.

Repregunta de presion: *¿Tendria sentido borrar automaticamente todos los hijos al borrar el padre en tu dominio? Responde por negocio, no por sintaxis.*

PREGUNTA 32

¿Que hace orphanRemoval=true y en que se diferencia de CascadeType.REMOVE?

Debe demostrar:

- orphanRemoval elimina un hijo cuando deja de pertenecer a la coleccion/asociacion propietaria segun el mapping.

- Cascade REMOVE se dispara al eliminar la entidad padre.
- Debe explicar cuando un hijo tiene ciclo de vida dependiente del padre.

Repregunta de presion: Si quitas un hijo de la lista del padre pero no borras el padre, ¿que esperas que ocurra con y sin orphanRemoval?

PREGUNTA 33 - CRITICA

Explica FetchType.LAZY y EAGER, incluyendo los defaults de @OneToMany y @ManyToOne.

Debe demostrar:

- @OneToMany es LAZY por defecto; @ManyToOne es EAGER por defecto en JPA, aunque es comun configurar LAZY explicitamente.
- Debe explicar costo de traer datos innecesarios y acceso fuera de sesion/transaccion.
- Debe diferenciar "relacion existe" de "relacion ya fue cargada".

Repregunta de presion: ¿Por que convertir todo a EAGER para evitar LazyInitializationException suele crear otros problemas?

PREGUNTA 34

En una relacion bidireccional, ¿por que conviene mantener sincronizados ambos lados en memoria?

Debe demostrar:

- Debe explicar metodos helper tipo addHijo/removeHijo que actualizan coleccion y referencia al padre.
- La base se controla por el lado propietario, pero el grafo de objetos debe quedar coherente.
- Debe predecir bugs de respuesta/validacion si un lado queda desactualizado.

Repregunta de presion: Escribe verbalmente que deberia hacer addHijo(hijo) paso a paso.

PREGUNTA 35 - CRITICA

¿Por que una relacion bidireccional puede generar recursion infinita al serializar JSON y como la evitas?

Debe demostrar:

- Padre contiene hijos y cada hijo vuelve a contener padre; Jackson puede recorrer un ciclo.
- La solucion preferible en APIs empresariales suele ser no exponer entidades directamente y usar DTOs con la forma exacta de respuesta.
- Puede conocer @JsonIgnore/@JsonManagedReference/@JsonBackReference, pero debe entender sus trade-offs.

Repregunta de presion: ¿Por que "poner @JsonIgnore hasta que deje de fallar" no sustituye un buen contrato DTO?

PREGUNTA 36

¿Que es el persistence context y que significa dirty checking?

Debe demostrar:

- Debe explicar que Hibernate mantiene entidades gestionadas dentro de un contexto y detecta cambios.
- En una transaccion, una entidad cargada y modificada puede generar UPDATE al flush/commit aunque no llames save() otra vez.
- Debe relacionar esto con identidad de objetos y transaccion.

Repregunta de presion: Carga una entidad, cambia un campo y no llamas save(). ¿Puede actualizarse la base? ¿Bajo que condiciones?

PREGUNTA 37

Diferencia los estados transient, managed, detached y removed de una entidad JPA.

Debe demostrar:

- Transient: nueva/no gestionada; managed: dentro del persistence context; detached: fue gestionada pero ya no; removed: marcada para eliminar.
- Debe vincular persist/merge/remove conceptualmente.
- Debe evitar creer que toda instancia @Entity esta siempre conectada a la base.

Repregunta de presion: ¿Que pasa si modificas una entidad detached fuera de transaccion?

PREGUNTA 38

¿Que es el problema N+1 y como podrias detectarlo y resolverlo?

Debe demostrar:

- Debe explicar una consulta inicial mas N consultas por relaciones.
- Debe mencionar logs SQL/profiling y soluciones segun caso: fetch join, EntityGraph, proyecciones, consultas DTO, batch/paginacion.
- Debe evitar responder "poner EAGER" como solucion universal.

Repregunta de presion: Si listar 100 padres dispara 101 consultas, ¿que te esta indicando?

PREGUNTA 39

Explica spring.jpa.hibernate.ddl-auto y por que "update" no es una estrategia profesional de migracion de produccion.

Debe demostrar:

- Debe conocer create/create-drop/update/validate/none a nivel conceptual.
- Debe explicar que migraciones controladas con Flyway/Liquibase permiten versionar esquema de forma reproducible.
- Debe separar comodidad de desarrollo de control de cambios productivo.

Repregunta de presion: Si dos desarrolladores cambian entidades de manera diferente y ambos usan update, ¿como garantizas un esquema reproducible?

PREGUNTA 40

Desde repository.save() hasta PostgreSQL: explica como JPA/Hibernate termina ejecutando SQL.

Debe demostrar:

- Spring Data delega a JPA/EntityManager; Hibernate traduce cambios/operaciones a SQL; JDBC envia sentencias al driver; PostgreSQL ejecuta y devuelve resultado.
- Debe poder activar/leer logs SQL y reconocer INSERT/SELECT/UPDATE/DELETE.
- Debe saber que JpaRepository no "es la base de datos".

Repregunta de presion: ¿Como demostrarías durante la defensa que un POST produjo realmente un INSERT y no solo un objeto en memoria?

7. Bloque D - DTO, record, servicios y repositorios

Objetivo: comprobar que el contrato HTTP esta separado de persistencia y que servicios/repositorios tienen responsabilidades claras.

PREGUNTA 41

¿Por que no conviene exponer tus @Entity directamente como contrato de la API?

Debe demostrar:

- Acopla contrato externo a esquema/persistencia, puede filtrar campos, generar ciclos, overfetching y romper clientes ante cambios internos.
- DTO permite controlar entrada/salida y evolucionar API de manera intencional.
- Debe diferenciar modelo de persistencia de contrato HTTP.

Repregunta de presion: Dame un campo de tu entidad que no deberia viajar necesariamente en la respuesta y explica por que.

PREGUNTA 42

Diferencia Request DTO y Response DTO. ¿Por que no siempre deben tener los mismos campos?

Debe demostrar:

- Request representa lo que el cliente puede enviar; Response lo que el servidor decide exponer.
- El id generado, timestamps o datos calculados pueden estar solo en Response; campos internos no deben exponerse.
- Debe hablar de validaciones especificas de entrada.

Repregunta de presion: En un PUT, ¿permitirías que el cliente cambie directamente el id o una fecha de auditoria? Justifica.

PREGUNTA 43 - CRITICA

¿Que es un record de Java y por que es util para DTOs?

Debe demostrar:

- Record declara un portador de datos compacto con componentes, constructor canonico y accesorios; es final y no usa setters tradicionales.
- Es muy adecuado para mensajes/DTOs inmutables por intencion.
- Debe reconocer que si un componente contiene un objeto mutable, la inmutabilidad no es profunda automaticamente.

Repregunta de presion: ¿Usarias record como @Entity JPA en un proyecto normal? Explica las incompatibilidades/limitaciones practicas y por que un DTO record si encaja mejor.

PREGUNTA 44

¿Donde y como conviertes Entity <-> DTO? Defiende tu estrategia de mapeo.

Debe demostrar:

- Puede mapear manualmente o con herramienta como MapStruct; lo importante es una frontera clara.
- Controller no deberia llenarse de conversiones complejas; el mapping puede vivir en un mapper/adaptador segun diseño.
- Debe evitar devolver relaciones enteras accidentalmente.

Repregunta de presion: Si agregas un campo nuevo al Response DTO, ¿que capas deberias revisar?

PREGUNTA 45 - CRITICA

¿Que responsabilidades pertenecen al Service/caso de uso y cuales NO?

Debe demostrar:

- Debe contener orquestacion y reglas de negocio de aplicacion, no detalles HTTP ni SQL concretos.
- Debe evitar un fat controller y tambien un service que solo copie llamadas sin ninguna responsabilidad si el caso no lo necesita.
- Debe decidir transacciones y coordinar puertos/repositorios.

Repregunta de presion: *Muestrame una regla de negocio real de tu proyecto y explica por que no debe vivir en el Controller.*

PREGUNTA 46

Diferencia un puerto de repositorio del dominio/aplicacion y un JpaRepository de Spring Data.

Debe demostrar:

- El puerto expresa lo que el nucleo necesita; el adaptador implementa ese contrato usando Spring Data/JPA.
- @Repository identifica un componente de persistencia y participa en traduccion de excepciones; los repositorios Spring Data son proxies administrados.
- Debe reconocer si en su version simplificada usa JpaRepository directamente y que acoplamiento implica.

Repregunta de presion: *Si quieres probar el servicio sin levantar PostgreSQL, ¿que ventaja te da tener un puerto de repositorio propio?*

PREGUNTA 47

Explica repository.findById(), Optional, consultas derivadas y @Query.

Debe demostrar:

- Optional modela ausencia; debe tratarse explícitamente.
- Spring Data puede derivar consultas desde nombres de metodos y tambien ejecutar JPQL/native queries con @Query.
- Debe diferenciar una consulta de repositorio de una regla de negocio.

Repregunta de presion: *¿Que diferencia hay entre findById(), getReferenceById() y una consulta custom en cuanto a intencion y acceso a datos?*

8. Bloque E - Validacion, pruebas y evolucion a microservicios

Objetivo: comprobar que el estudiante comprende robustez, calidad y evolucion arquitectonica mas alla de que el CRUD "funcione".

PREGUNTA 48

Separa tres niveles de validacion: DTO/Bean Validation, reglas de negocio y constraints de PostgreSQL.

Debe demostrar:

- Bean Validation protege forma/valores basicos de entrada; servicio/dominio protege reglas contextuales; DB garantiza integridad final/concurrencia con PK/FK/UNIQUE/NOT NULL.
- Debe explicar que una validacion duplicada en capas puede ser intencional, no redundancia inutil.
- Debe mapear conflictos de unicidad/integridad a una respuesta coherente, por ejemplo 409 cuando corresponde.

Repregunta de presion: *Dos peticiones simultaneas pasan la validacion "no existe" y luego intentan guardar el mismo valor unico. ¿Que capa termina resolviendo la carrera?*

PREGUNTA 49

¿Como probarias Controller, Service y Repository sin mezclar responsabilidades?

Debe demostrar:

- Controller: contrato HTTP, validacion, status, serializacion usando pruebas web; Service: reglas con dependencias simuladas/fakes; Repository: mapping/queries con una base de prueba.
- Debe diferenciar unit test e integration test.
- Debe explicar que mockear todo puede ocultar problemas de JPA/SQL y que una prueba de repositorio necesita infraestructura realista.

Repregunta de presion: *Dame una prueba que detectaria un error en mappedBy que una prueba unitaria del service nunca detectaria.*

PREGUNTA 50 - CRITICA

Defiende por que un monolito hexagonal modular es una base muy solida para empezar y como facilita un salto futuro a microservicios.

Debe demostrar:

- Debe explicar que reduce complejidad operacional inicial frente a microservicios y, a la vez, fuerza limites, responsabilidades y dependencias claras.
- Puertos/adaptadores y modulos bien aislados permiten reemplazar infraestructura y extraer capacidades con menos impacto.

- Debe aclarar que no es una garantia automatica: para extraer un microservicio se necesitan fronteras de negocio reales, contratos, independencia de datos, observabilidad, despliegue y manejo de consistencia distribuida.
- Debe poder elegir un modulo de su proyecto y describir que moveria, que API/contrato dejaria y como separaria su persistencia.

Repregunta de presion: Si dos modulos comparten las mismas tablas, llaman directamente a repositorios del otro y tienen ciclos de dependencia, ¿estan realmente "listos para microservicios" aunque esten en paquetes separados?

9. Cinco desafios practicos adicionales para una defensa realmente exigente

Estos desafios no sustituyen las 50 preguntas. Sirven para comprobar que el estudiante domina el codigo y no solo el discurso.

Desafio 1 - Trazado en vivo

Elegir un POST real. El estudiante debe abrir Controller, DTO/record, Service/UseCase, mapper, Repository y Entity. Debe narrar el flujo sin ejecutar todavia. Luego ejecutar la llamada y comprobar en logs/DB que su prediccion fue correcta.

Desafio 2 - Romper una anotacion

Pedir que explique que ocurriria si se elimina mappedBy, @Valid, @Service o @JoinColumn. No necesariamente modificar el codigo; debe predecir el efecto y justificarlo.

Desafio 3 - Error intencional

Enviar un payload invalido, un id inexistente y un dato duplicado. El estudiante debe anticipar status, capa que detecta el problema y estructura de respuesta.

Desafio 4 - SQL y cardinalidad

Crear un padre con hijos, consultar tablas y mostrar PK/FK. Luego borrar o desvincular un hijo y explicar exactamente que SQL espera segun cascade/orphanRemoval.

Desafio 5 - Extraccion futura

Pedir al estudiante que elija una funcionalidad y describa como la convertiria en microservicio sin rediseñar todo el sistema: contrato, datos, dependencias, transaccion y comunicacion.

10. Senales de que el estudiante todavia NO esta preparado

- Puede crear el CRUD, pero no sabe explicar que sucede entre Controller y PostgreSQL.
- Dice que Swagger "es el backend" o que Postman "guarda en la base".
- Confunde Entity con DTO o usa ambos terminos como sinonimos.
- No sabe en que tabla se encuentra la foreign key de su uno-a-muchos.
- No sabe quien es el lado propietario ni que significa mappedBy.
- Usa CascadeType.ALL, EAGER o @JsonIgnore como recetas sin justificar consecuencias.
- No puede explicar por que usa record ni que diferencia tiene frente a una clase mutable.
- No distingue reglas de validacion, reglas de negocio y constraints de PostgreSQL.
- No sabe que status HTTP devuelve cada escenario de error.
- No puede dibujar el flujo POST/GET/PUT/DELETE sin mirar apuntes.
- Su Controller accede directamente a JPA y no puede explicar el acoplamiento creado.
- Afirma que por usar paquetes separados el sistema ya esta preparado automaticamente para microservicios.

11. Checklist final de preparacion del estudiante

- ☐ Puedo explicar el flujo completo de POST, GET, PUT y DELETE usando mi propio codigo.
- ☐ Puedo explicar todas las anotaciones que aparecen en mis clases principales.
- ☐ Puedo dibujar mis dos tablas y su relacion PK/FK sin consultar el codigo.
- ☐ Puedo explicar lado propietario, mappedBy, @JoinColumn, cascade, orphanRemoval y fetch.
- ☐ Puedo justificar por que uso DTO/record y por que no expongo entidades directamente.
- ☐ Puedo explicar que hace mi Service/caso de uso y que hace mi Repository.
- ☐ Puedo explicar donde empieza y termina una transaccion y que hace Hibernate dentro de ella.
- ☐ Puedo diferenciar 400, 404, 409 y 500 con ejemplos de mi proyecto.
- ☐ Puedo demostrar con logs o SQL que una operacion realmente llego a PostgreSQL.

☐ Puedo explicar por que el monolito hexagonal modular facilita una futura separacion en microservicios sin afirmar que la hace automatica.

MENSAJE AL DOCENTE

El objetivo no es que el estudiante memorice Spring Boot. El objetivo es que pueda leer su propio backend como un sistema: reconocer fronteras, responsabilidades, contratos, transformaciones de datos, decisiones de persistencia y consecuencias. Cuando pueda explicar un request completo sin saltos y defender por que cada pieza existe, esta preparado para la defensa oral.